

Lab 0: Development Environment Setup — macOS

Standalone share copy — self-contained; no course repository links required.

OS: macOS (Apple Silicon or Intel)

Primary IDE: IntelliJ IDEA Community Edition

Optional IDE: VS Code (if you already prefer it)

Install tools on **your Mac**. Shared cloud services are not needed until Week 4+.

What you install

Tool	Target
IntelliJ IDEA Community	Latest stable (primary)
VS Code (optional)	Latest stable + Extension Pack for Java
Temurin OpenJDK	21 LTS
Maven	3.9.x
Git	2.x (often preinstalled / Xcode CLT)

Steps

Step 1 — Install IntelliJ IDEA Community (primary)

1. Download **Community Edition** for macOS: <https://www.jetbrains.com/idea/download/>
2. Open the **.dmg** and drag IntelliJ to Applications.
3. Launch once and finish the first-run wizard.

Expected: Welcome screen opens.

Step 2 — Optional: Install VS Code

Only if you already prefer VS Code:

1. Install from <https://code.visualstudio.com/>
2. Extensions: **Extension Pack for Java**.

Step 3 — Install Temurin JDK 21 and set **JAVA_HOME**

Homebrew (recommended):

```
brew install --cask temurin@21
```

Add to **~/.zshrc** (default shell on modern macOS):

```
export JAVA_HOME=$(/usr/libexec/java_home -v 21)
export PATH="$JAVA_HOME/bin:$PATH"
```

Then:

```
source ~/.zshrc
java -version
javac -version
echo $JAVA_HOME
```

Expected: Versions show **21.x.x**.

If it fails: `java_home -V` lists installed JDKs — pick 21. Avoid leaving Java 8/11 first on **PATH**.

Step 4 — Install Maven 3.9.x and Git

```
# Git (if missing)
xcode-select --install # or: brew install git

# Maven
brew install maven
mvn -version
git --version
```

Expected: Maven **3.9.x** (or latest Homebrew 3.9.x line) using Java **21**.

Step 5 — Create `java-bootcamp` workspace

Where to run this: Open **Terminal.app** (Applications → Utilities → Terminal), or iTerm if you use it. Do **not** use IntelliJ's terminal yet — that comes in Step 6 after you open the folder. You can start from any directory; the commands create `~/java-bootcamp` under your home folder (for example `/Users/<you>/java-bootcamp`).

This layout matches what you use in every later lab: hands-on code under `examples/`, Lab 0 evidence under `notes/screenshots/`. Course lab handouts are provided separately by your instructor — you do **not** need a top-level `labs/` folder in this workspace.

```
mkdir -p ~/java-bootcamp/examples ~/java-bootcamp/notes/screenshots
cd ~/java-bootcamp
ls
ls notes
```

Expected:

```
java-bootcamp/  
  examples/          # HelloJava next; later labs: lab1-answers, labN-crm, ...  
  notes/  
    screenshots/     # Pass-criteria screenshots (redact secrets)
```

Step 6 — Open workspace in IntelliJ

1. **File** → **Open...** → `~/java-bootcamp`.
2. Trust the project if prompted.
3. **File** → **Project Structure** → **Project** → SDK **21**, language level **21**.
4. **View** → **Tool Windows** → **Terminal** → `java -version`.

Optional VS Code: File → Open Folder... → same path.

Step 7 — HelloWorld sources

```
cd ~/java-bootcamp  
mkdir -p examples/HelloJava/src examples/HelloJava/out
```

Create `examples/HelloJava/src/HelloJava.java` (class name must match; macOS is case-sensitive):

```
public class HelloJava {  
    public static void main(String[] args) {  
        System.out.println("Hello Java Bootcamp!");  
    }  
}
```

Step 8 — Run from IntelliJ terminal

```
cd ~/java-bootcamp/examples/HelloJava  
javac -d out src/HelloJava.java  
java -cp out HelloJava
```

Expected:

```
Hello Java Bootcamp!
```

Step 9 — Run with IntelliJ green arrow

1. Open `HelloJava.java`.
2. Right-click `src` → **Mark Directory as** → **Sources Root** if needed.
3. Green ▶ beside `main` → **Run 'HelloJava.main()'**.

Step 10 — Git + identity (for later labs)

- 1. Create or sign in to a **GitHub** account (website: github.com).
- 2. Set Git to your **display name** and your GitHub **noreply email** (recommended — avoids push errors if your personal email is private on GitHub).

Find your noreply email: GitHub → **Settings** → **Emails** → enable **Keep my email addresses private** → copy the address shown, e.g. `{id}+{username}@users.noreply.github.com`.

```
git config --global user.name "Your Name"
git config --global user.email "12345678+yourusername@users.noreply.github.com"
git config --global --list
git --version
```

Expected: `user.name` and `user.email` appear in the list; `git --version` shows **2.x**.

If push fails with GH007 (“private email address”): your commit used a personal email address. Re-run Step 10 with the **noreply** email from GitHub Settings → Emails, or adjust email privacy on that same page.

Keep personal verification screenshots under `~/java-bootcamp/notes/screenshots/` only — do not publish them.

Pass criteria (macOS)

```
java -version
javac -version
mvn -version
git --version
echo $JAVA_HOME
cd ~/java-bootcamp && pwd && ls
```

Mark each row **Pass** or **Fail** in your own notes.

#	Confirm	Your notes
1	Java / javac 21 (Temurin)	Pass / Fail
2	Maven 3.9.x on Java 21	Pass / Fail
3	Git works; <code>user.name</code> / <code>user.email</code> set	Pass / Fail
4	<code>JAVA_HOME</code> points at JDK 21	Pass / Fail
5	Workspace <code>~/java-bootcamp</code> with <code>examples/</code> and <code>notes/screenshots/</code>	Pass / Fail
6	HelloWorld prints from terminal	Pass / Fail
7	HelloWorld runs via IntelliJ green arrow	Pass / Fail

#	Confirm	Your notes
8	(Optional) VS Code opens the same folder	Pass / Fail

Do not start Lab 1 until every Pass criteria row is Pass in your notes.

When complete, continue to **Lab 1** in your course materials.